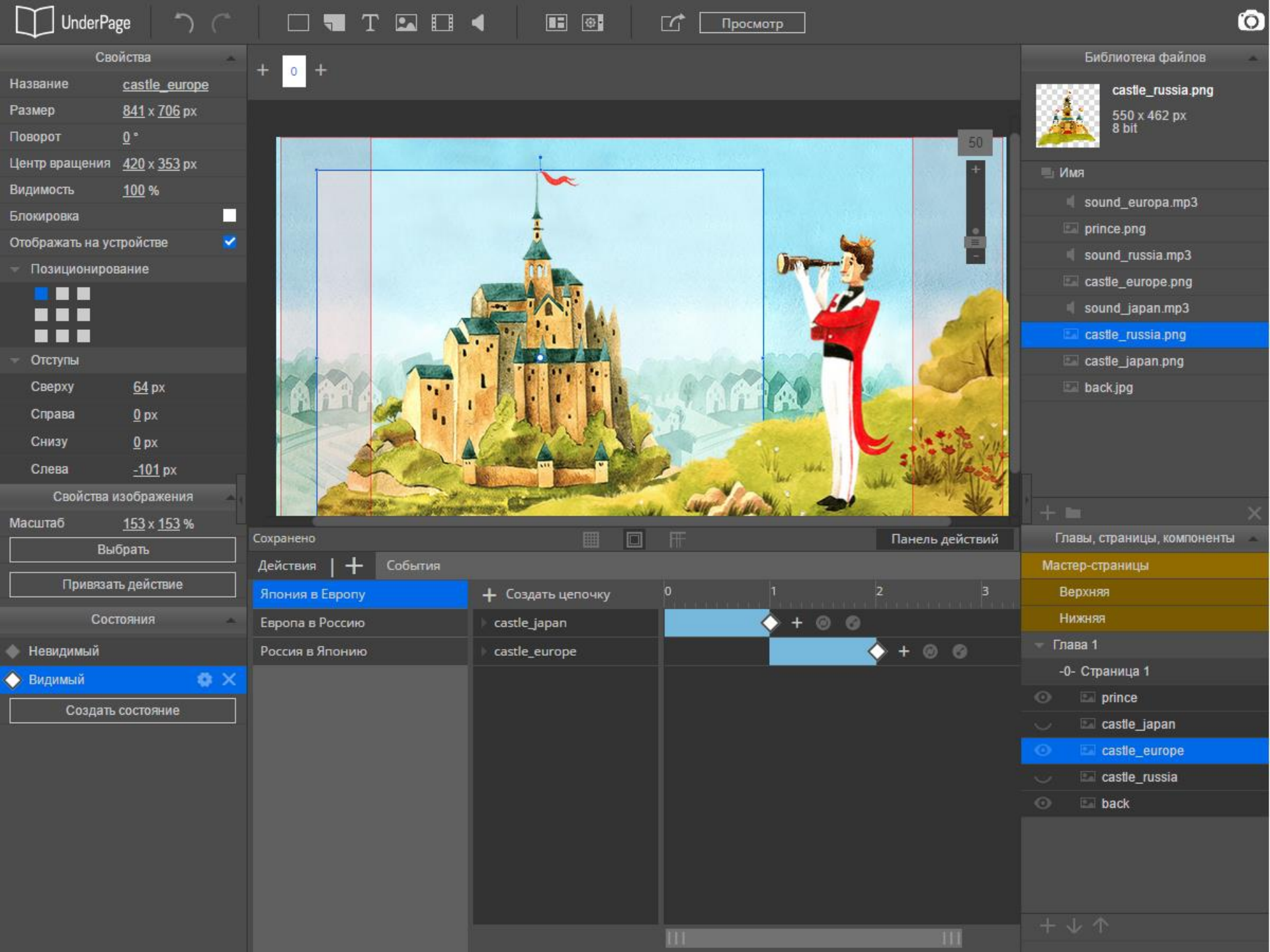


UnderPage



HTML5 Web Components: следующий шаг к модульности вашего проекта

Андрей Рахманов, Enaza



Свойства

Название castle_europe

Размер 841 x 706 px

Поворот 0 °

Центр вращения 420 x 353 px

Видимость 100 %

Блокировка ☐

Отображать на устройстве ☒

Позиционирование

☒ ☐ ☐

☐ ☐ ☐

☐ ☐ ☐

Отступы

Сверху 64 px

Справа 0 px

Снизу 0 px

Слева -101 px

Свойства изображения

Масштаб 153 x 153 %

Выбрать

Привязать действие

Состояния

Невидимый

Видимый

Создать состояние

Сохранено

Действия



События

Япония в Европу

Европа в Россию

Россия в Японию

+ Создать цепочку

castle_japan

castle_europe

0

1

2

3

+

+

Панель действий

Библиотека файлов



castle_russia.png

550 x 462 px
8 bit

Имя

sound_europa.mp3

prince.png

sound_russia.mp3

castle_europe.png

sound_japan.mp3

castle_russia.png

castle_japan.png

back.jpg



Главы, страницы, компоненты

Мастер-страницы

Верхняя

Нижняя

Глава 1

-0- Страница 1

prince

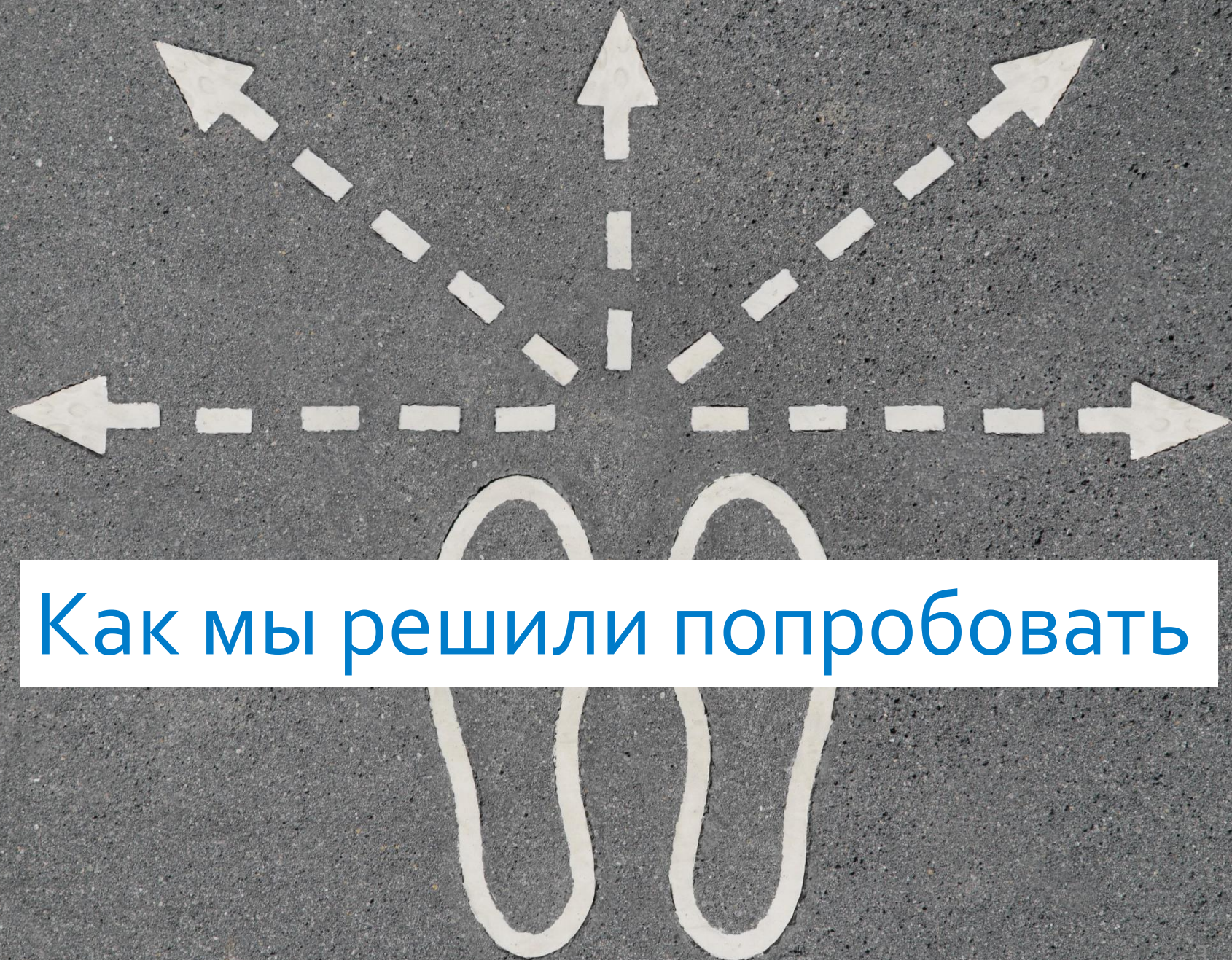
castle_japan

castle_europe

castle_russia

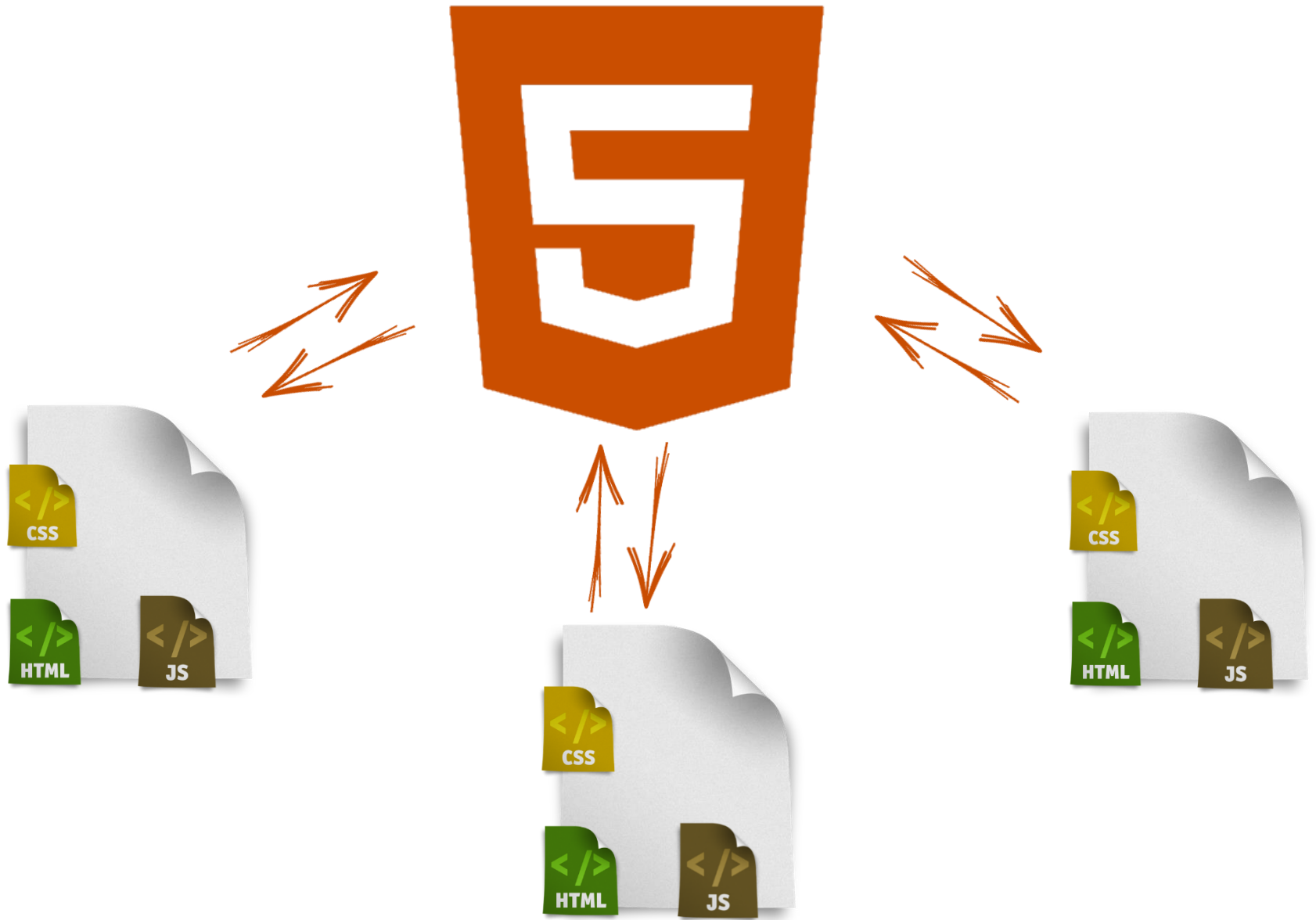
back





Как мы решили попробовать

Single-Page App









Код победил!

```
<style>
  #map {
    height: 400px;
  }
</style>

<div id="map"></div>

<script src="http://maps.googleapis.com/maps/api/js?callback=mapReady"></script>
<script>
  var marker = null;

  function getCurrentLocation(callback) {
    navigator.geolocation.watchPosition(callback);
  }

  function addMarker(opts, info) {
    var marker = new google.maps.Marker(opts);

    var infoWindow = new google.maps.InfoWindow({content: info});

    google.maps.event.addListener(marker, 'click', function() {
      infoWindow.open(opts.map, marker);
    });

    return marker;
  }

  function mapReady() {
    var container = document.querySelector('#map');
    //...
  }
  //...
```

```
<style>
  #map {
    height: 400px;
  }
</style>

<div id="map"></div>

<script src="http://maps.googleapis.com/maps/api/js?callback=mapReady"></script>
<script>
  var marker = null;

  function getCurrentLocation(callback) {
    navigator.geolocation.watchPosition(callback);
  }
}
```

<google-map></google-map>

```
var infoWindow = new google.maps.InfoWindow({content: info});

google.maps.event.addListener(marker, 'click', function() {
  infoWindow.open(opts.map, marker);
});

return marker;
}

function mapReady() {
  var container = document.querySelector('#map');
  //...
}
//...
```



BACKBONE.JS

Поиск подходящего решения

- Плохо сочетать Backbone.js с другими фреймворками
- Надо использовать native-реализацию или библиотеку



Составные части Web Components

- Templates
- HTML Imports
- Custom Elements
- Shadow DOM

Templates



HTML Imports



Custom Elements



Shadow DOM



Native: Пока весьма печально...

Templates



HTML Imports



Custom Elements



Shadow DOM



Polyfills: Уже лучше!

Что нам предлагают на текущий момент?



Native



Bosonic



X-Tag (Mozilla)

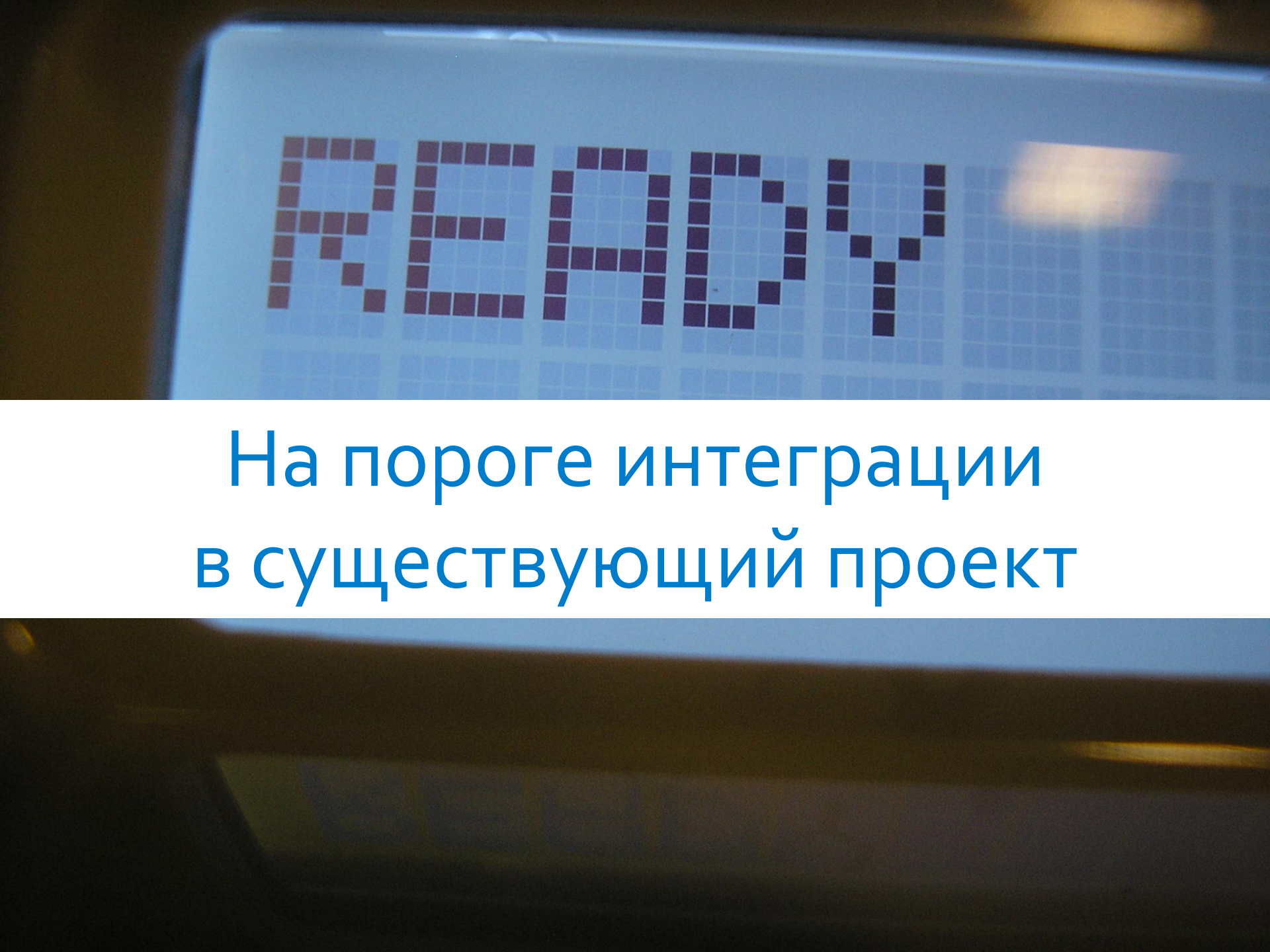


Polymer (Google)

Мы выбрали Polymer

- Декларативное описание элементов
- Удобный «синтаксический сахар» над нативной реализацией



A close-up photograph of a digital display, likely from a video game or a retro computer terminal. The display is dark, and the word "READY" is shown in a bright, pixelated, blocky font. The letters are composed of small squares, giving it a digital, low-resolution appearance. The background is dark and out of focus, with some light reflecting off the surface of the display.

READY

На пороге интеграции
в существующий проект

Стек технологий



Стек технологий



Стек технологий



Как делаем мы

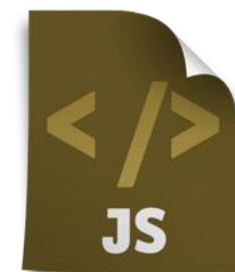
iade
Node Template Engine



stylus



CommonJS
browserify



Чего ожидает от нас Polymer

```
<dom-module>
```

```
  <style>
```



```
  </style>
```

```
  <template>
```



```
  </template>
```

```
</dom-module>
```

```
<script>
```



```
</script>
```

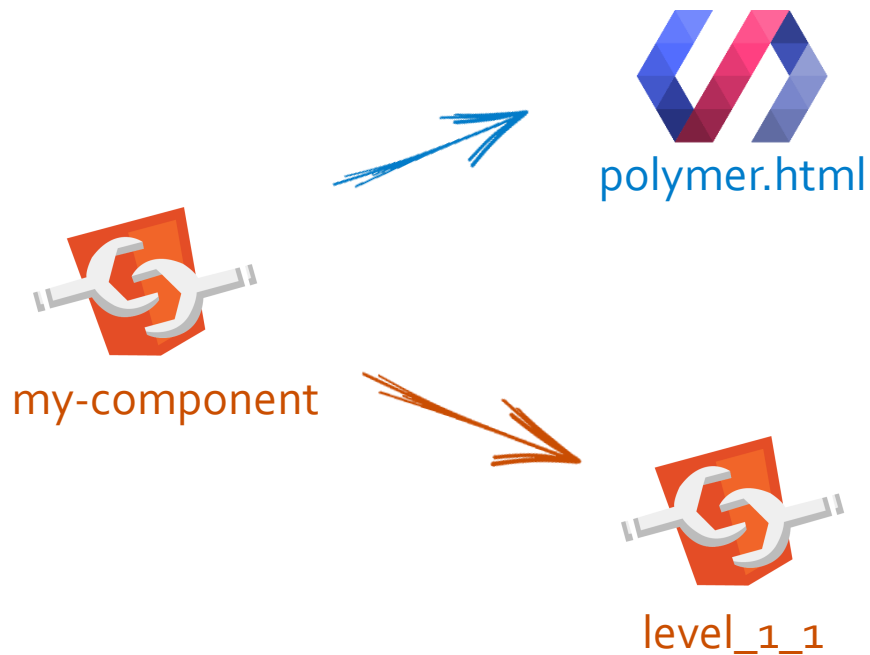


Файл с компонентом

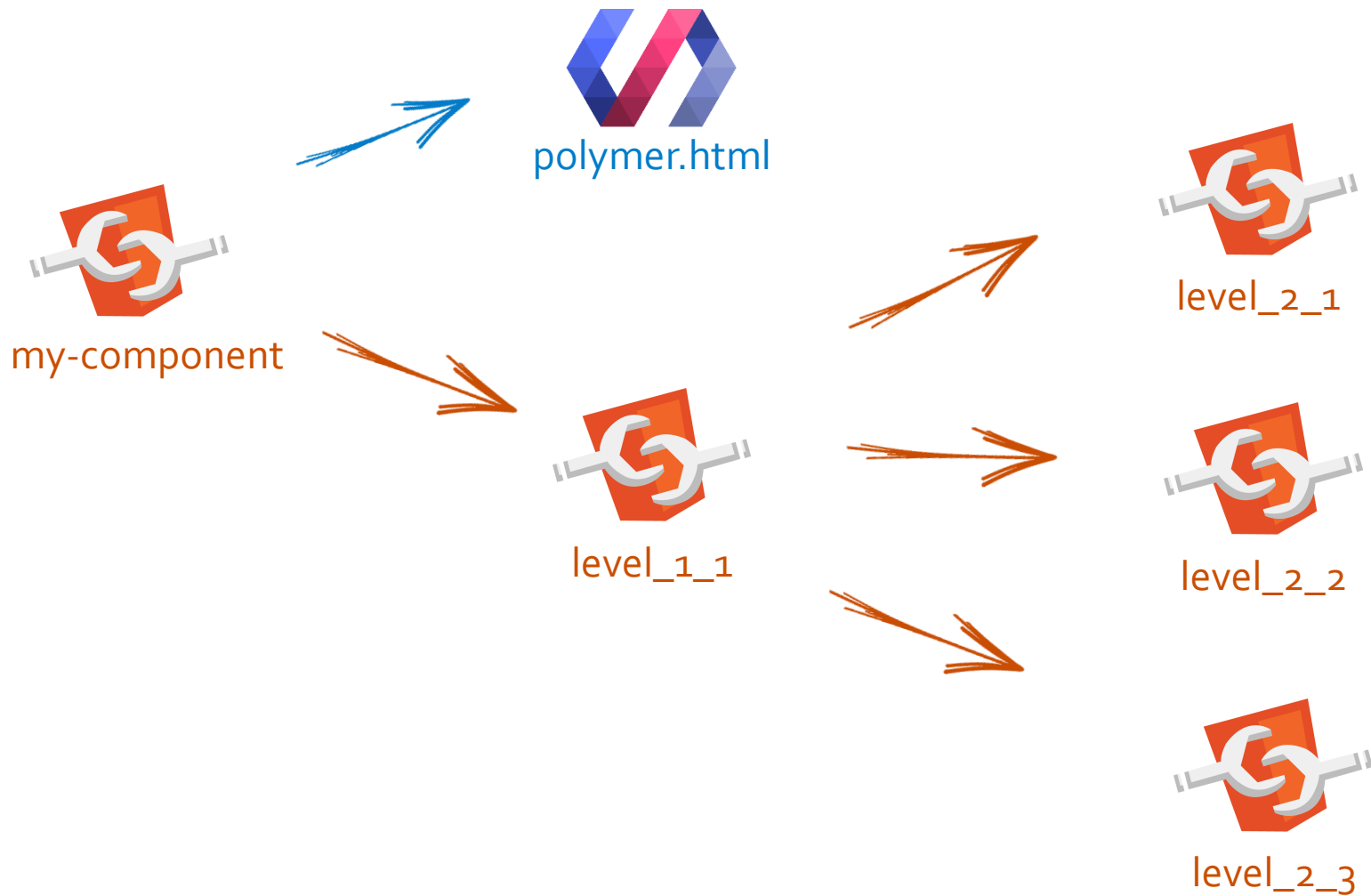
Подключаем один компонент



Подключаем один компонент



Подключаем один компонент



```
<link rel="import" href="simple_component.html">
```

а за ним пойдут...

```
<link rel="import" href="polymer.html">
```

```
<link rel="import" href="level_1_1.html">
```

```
<link rel="import" href="level_2_1.html">
```

```
<link rel="import" href="level_2_2.html">
```

```
<link rel="import" href="level_2_3.html">
```

1 import == 6 запросов

Out-of-box: Polymer Vulcanize

Реализация:

```
var vulcan = require('vulcanize');

vulcan.setOptions({
  /* конфигурация сборки */
});

vulcan.process(target, function(err, inlinedHtml) {
  /* запись готового html в файл all_in_one.html */
});
```

Использование:

```
<link rel="import" href="all_in_one.html">
```

Можно ли ограничиться минимумом сетевых запросов?

vendor.js



app.js



Как положить всё в скрипты

webcomponents-lite.js



прочие библиотеки



polymer.html



vendor.js

JS

<наши компоненты>.html

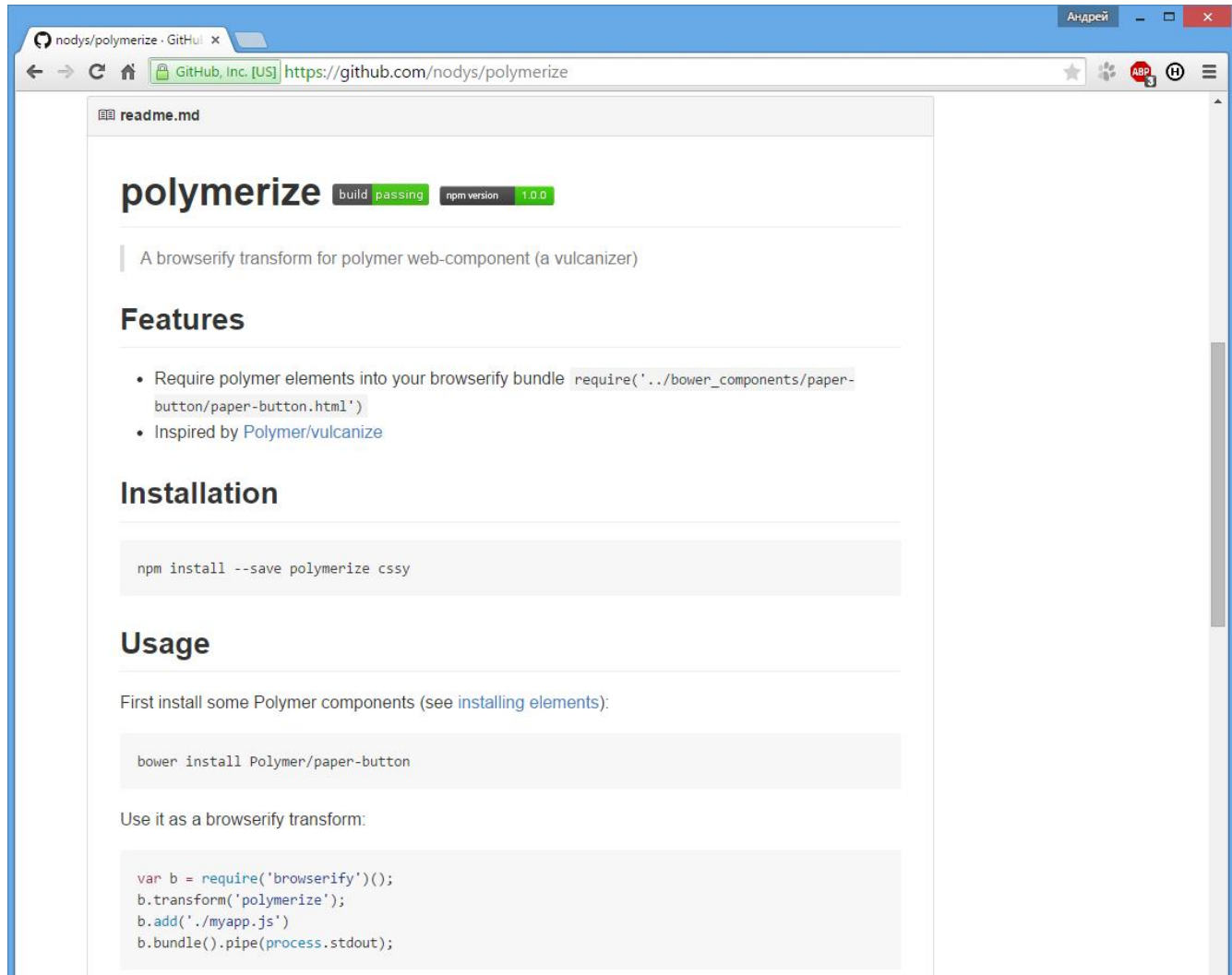


app.js

JS

Очевидное решение:

Browserify Transform (polymerize)



nodys/polymerize · GitHub x

← → ↻ 🏠 GitHub, Inc. [US] https://github.com/nodys/polymerize

📖 readme.md

polymerize

build passing npm version 1.0.0

A browserify transform for polymer web-component (a vulcanizer)

Features

- Require polymer elements into your browserify bundle `require('../bower_components/paper-button/paper-button.html')`
- Inspired by [Polymer/vulcanize](#)

Installation

```
npm install --save polymerize cssy
```

Usage

First install some Polymer components (see [installing elements](#)):

```
bower install Polymer/paper-button
```

Use it as a browserify transform:

```
var b = require('browserify')();
b.transform('polymerize');
b.add('./myapp.js')
b.bundle().pipe(process.stdout);
```

Использовать - удобно

```
require( 'paper-button.html' )
```

Но реализация - неудачная

1) Требуется fix для Polymer:

```
this.queue(src
  .replace(/\w+\.(PolymerExpressions|esprima)(\s*=)/g, 'window.$1$2')
)
```

2) Требуется HTML Inject:

```
// src - результат парсинга файла с компонентами
```

```
// внедрение html
```

```
result.push('head.insertAdjacentHTML("beforeend",' +
  JSON.stringify(src.head)+'');')
```

```
result.push('var root = body.appendChild(document.createElement("div"));');
result.push('root.innerHTML = ' + JSON.stringify(src.body) + '');
```

Что будет, если регистрировать веб-компонент из скрипта?

```
var el = document.createElement('div');  
el.innerHTML = '\n  <dom-module is="name-tag">\n    <template>\n      Hello <span>{{name}}</span>\n    </template>\n  </dom-module>';
```

Результат

- Такой код неудобно поддерживать
- Нерешенный вопрос с подключением `polymer.html`

Итак, Vulcanize

component_2.html

component_3.html

component_1.html

styles.css

polymer.html

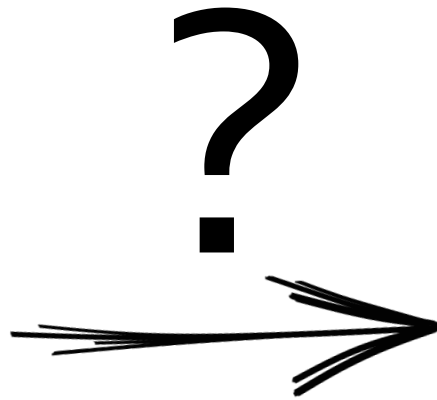
script.js



Варианты подключения



components.html



index.html

Внедрение в index.html

```
<html>
  <!-- содержимое index.html -->

  <script>
    // код библиотеки Polymer
  </script>

  <dom-module id="custom-component">
    <!-- код компонента -->
  </dom-module>
  <script>
    // логика компонента
  </script>

  <!-- подключение других компонентов -->
</html>
```



Меньше
запросов



Сборка
не кэшируется

HTML Import сборки

```
<html>  
  <!-- содержимое index.html -->  
  
  <link rel="import" href="components.html">  
</html>
```



1 лишний запрос
не так страшен



Сборка
кэшируется

Резюме

- Подключение компонентов в сборки JS – нестабильное решение
- Vulcanize снижает число сетевых запросов и решает проблему

Хочется удобства:
подружим Polymer, Jade,
Stylus и Browserify



jade
Node Template Engine



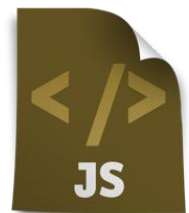
stylus



JS



CommonJS
browserify

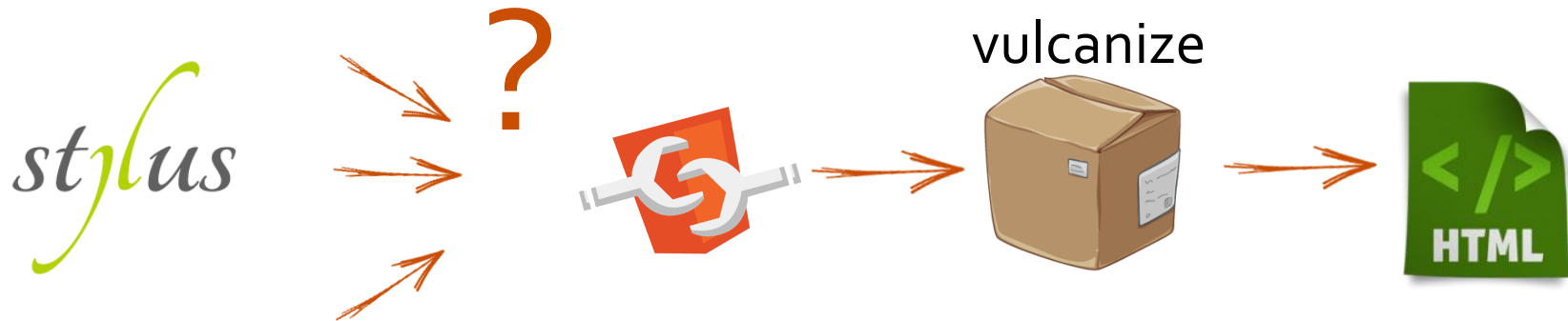


jade
Node Template Engine

stylus

JS







Вот кто нам поможет!

Gulp Task: Jade -> HTML

```
var searchTemplate = dirname + '/*.jade';
```

```
gulp.task(taskId, function () {  
  return gulp.src(searchTemplate)  
    .pipe(buffer())  
    .pipe(jade({}))  
    .pipe(gulp.dest(dirname));  
});
```

Gulp Task: Stylus -> CSS

```
var searchTemplate = dirname + '/*.styl';

gulp.task(taskId, function () {
  return gulp.src(searchTemplate)
    .pipe(buffer())
    .pipe(stylus({}))
    .pipe(gulpif(env == 'production', minifyCSS()))
    .pipe(gulp.dest(dirname));
});
```

Gulp Task: склеим 3 файла в 1

```
var searchTemplates = [  
  dirname + '/prebuild-*.html',  
  dirname + '/prebuild-*.css',  
  dirname + '/prebuild-*.js'  
];  
  
gulp.task(taskId, function () {  
  return gulp.src(searchTemplates)  
    // сборка в нужном порядке  
    .pipe(order(['*.html', '*.css', '*.js']))  
    .pipe(buffer())  
    .pipe(concat(resultname))  
    .pipe(gulp.dest(dirname));  
});
```

iade
Node Template Engine

stylus

JS



first.html



iade
Node Template Engine

stylus

JS



second.html



iade
Node Template Engine

stylus

JS



third.html



iade
Node Template Engine

stylus

JS

first.html



register_components.html

iade
Node Template Engine

stylus

JS

second.html



html imports:

first.html
second.html
third.html

vulcanize



iade
Node Template Engine

stylus

JS

third.html



```
<!-- подключение библиотеки Polymer -->
<link rel="import"
  href="./bower_components/polymer/polymer.html">

<!-- динамический реестр используемых компонентов -->
<link rel="import"
  href="./client/wc-up-input-autoresize/up-input-autoresize.html">

<link rel="import"
  href="./client/wc-up-textarea-count/up-textarea-count.html">

<link rel="import"
  href="./client/wc-up-select/up-select.html">
```

Этот реестр формирует Gulp

Управление JS-зависимостями

`<script src="...">` **VS** `require('module')`

`<script src="...">` **VS** `require('module')`

- Одного JS-файла в компоненте не хватает
- Почему не использовать `<script src="...">`?
CommonJS удобнее!
- Polymer не поддерживает CommonJS.
Как собрать самим?



* jQuery *
* plugins *
* lodash *
* Backbone *



components.html

* jQuery *
не дублируется

Как сделать так, чтобы компоненты писались сами?



```
npm install -g generator-poly-jadestyl
```

```
yo poly-jadestyl add my-web-component
```





```
dom-module(id="my-web-component")  
  template  
    p Test content
```



```
Polymer({  
  is: 'my-web-component',  
  
  ready: function () {  
  }  
})
```



```
:host  
  background red
```

Резюме

- Web Components успешно интегрируются с имеющимися инструментами
- Gulp способен решить множество задач по сборке компонентов
- Создание нового компонента сводится к одной строке кода для генератора

Где применить
веб-компоненты?



Декомпозиция Single-Page App

На низком уровне

Расширение стандартных элементов управления

На высоком уровне

Переиспользуемые логические блоки

Низкий уровень

Расширение стандартных элементов управления

Расширение стандартных элементов управления

- `<input is="up-autoresizable" />`
 - автоподстройка длины
- `<input is="up-colorpicker" />`
 - выбор цвета с альфа-каналом
 - указание цвета числом
- `<up-textarea-count></...>`
 - подсчет числа оставшихся символов
- `<up-select></...>`
 - иерархия
 - запрещенные значения в списке

```
<input is="up-autoresizable"/>
```

```
var $ = require('jquery');           | зависимости через  
    require('jquery.autosize.input'); | Browserify
```

```
Polymer({  
  is: 'up-autosizable',               | наследование  
  extends: 'input',                  | от input  
  
  // момент добавления в DOM  
  attached: function () {            | подключение  
    $this.autosizeInput();           | плагина  
  },  
  
  detached: function () {            | отключение  
    $this.autosizeInput('destroy');  | плагина  
  }  
})
```

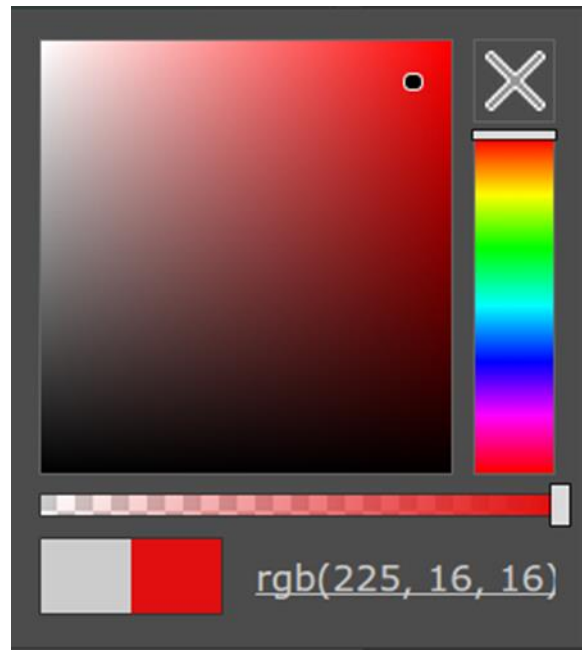
`<input is="up-colorpicker"/>`

```
Polymer({  
  is: 'up-colorpicker',  
  extends: 'input',  
  
  //... properties  
  
  ready: function () {  
    $(this).spectrum();  
  },  
  
  // примерить цвет  
  handleMove: function () {  
    $(this).trigger('preview', color);  
  }  
})
```

| наследование
| от input

| кастомное
| событие

```
<input is="up-colorpicker" alpha="1" />
```



<up-textarea-count>

```
Polymer({  
  is: 'up-textarea-count',  
  
  get value() {  
    return this.$.textarea.value;  
  },  
  
  set value(value) {  
    this.$.textarea.value = value;  
  },  
  
  listeners: {  
    'textarea.keyup': 'calcLength',  
    'textarea.change': 'calcLength'  
  },  
  
  calcLength: function () {  
    this.counter = this.value.length;  
  }  
  // ... прочая логика  
})
```

| Public API
| для взаимодействия
|
|
|

```
<up-textarea-count maxlength="140" rows="10" cols="20">  
  FrontendConf 2015  
</up-textarea-count>
```

Сопроводительный текст

FrontendConf 2015

123

<up-select>

```
Polymer({  
  is: 'up-select',  
  
  properties: {  
    // ...  
    multiple: {  
      type: Boolean  
      value: false  
    },  
    placeholder: {  
      type: String,  
      value: 'Выберите значение',  
    },  
    items: {  
      type: Array,  
      value: []  
    }  
  },  
  
  // логика изменения  
  // выбранного значения  
})
```

| свойства
| для биндинга
|
|
|
|
|
|
|

```
<up-select  
  placeholder="Выберите устройство"  
  items="{{devices}}">  
</up-select>
```

Устройство ?	ASUS Fonepad 7 (FE170CG) ▼	Размер	Во весь экран ▼
Android			
	ASUS Fonepad 7 (FE170CG)	12x7	
	HTC Nexus 9	4x3	
	Samsung GALAXY Tab 4	8x5	
iOS			
	iPad 2,3,4/iPad Air	4x3	
	iPad mini/iPad mini 2	4x3	

Высокий уровень

Переиспользуемые логические блоки

Переиспользуемые логические блоки

<внешний блок (атрибуты...)>

<внутренний блок 1 (атрибуты...)>

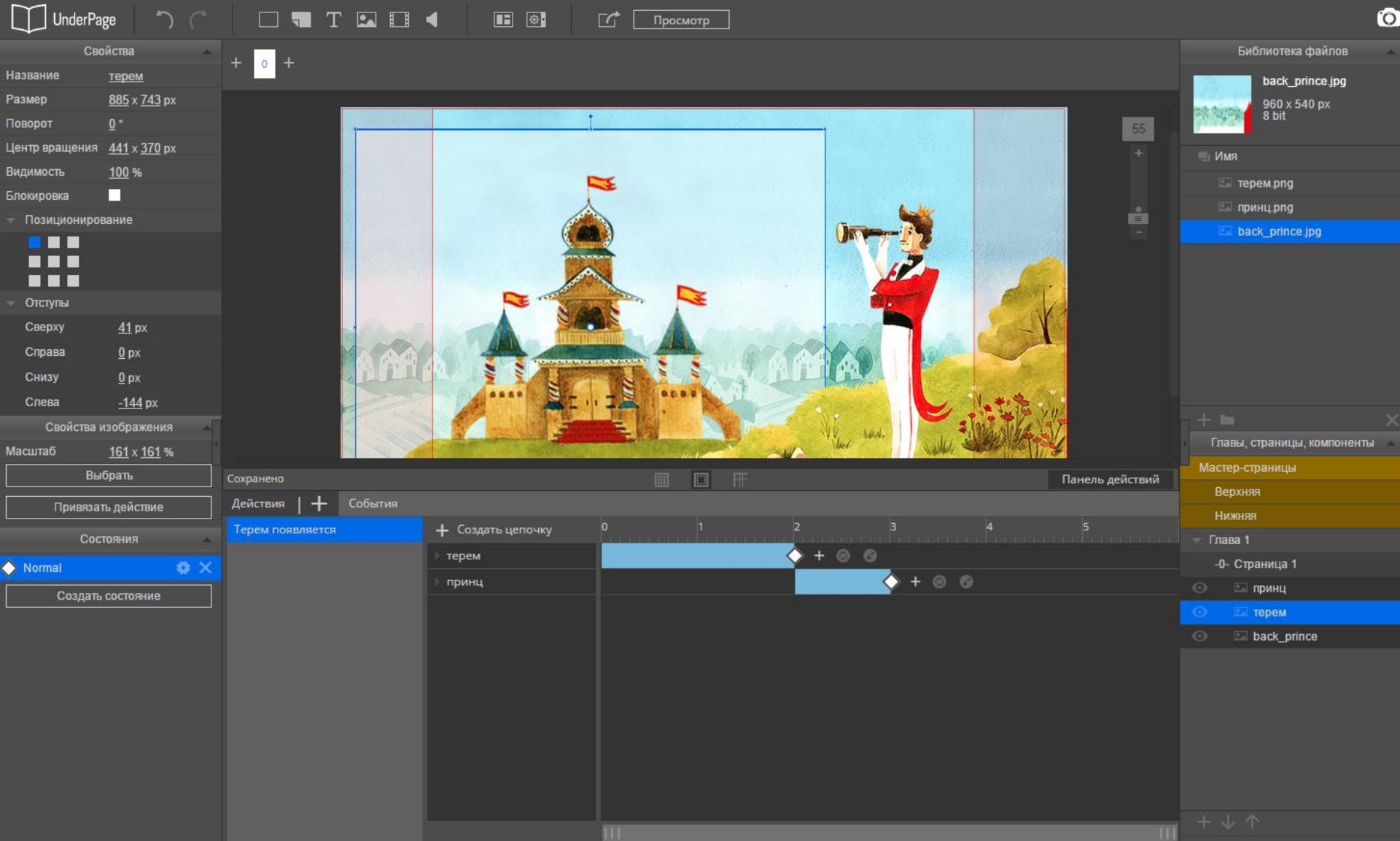
<поле 1>

<поле 2>

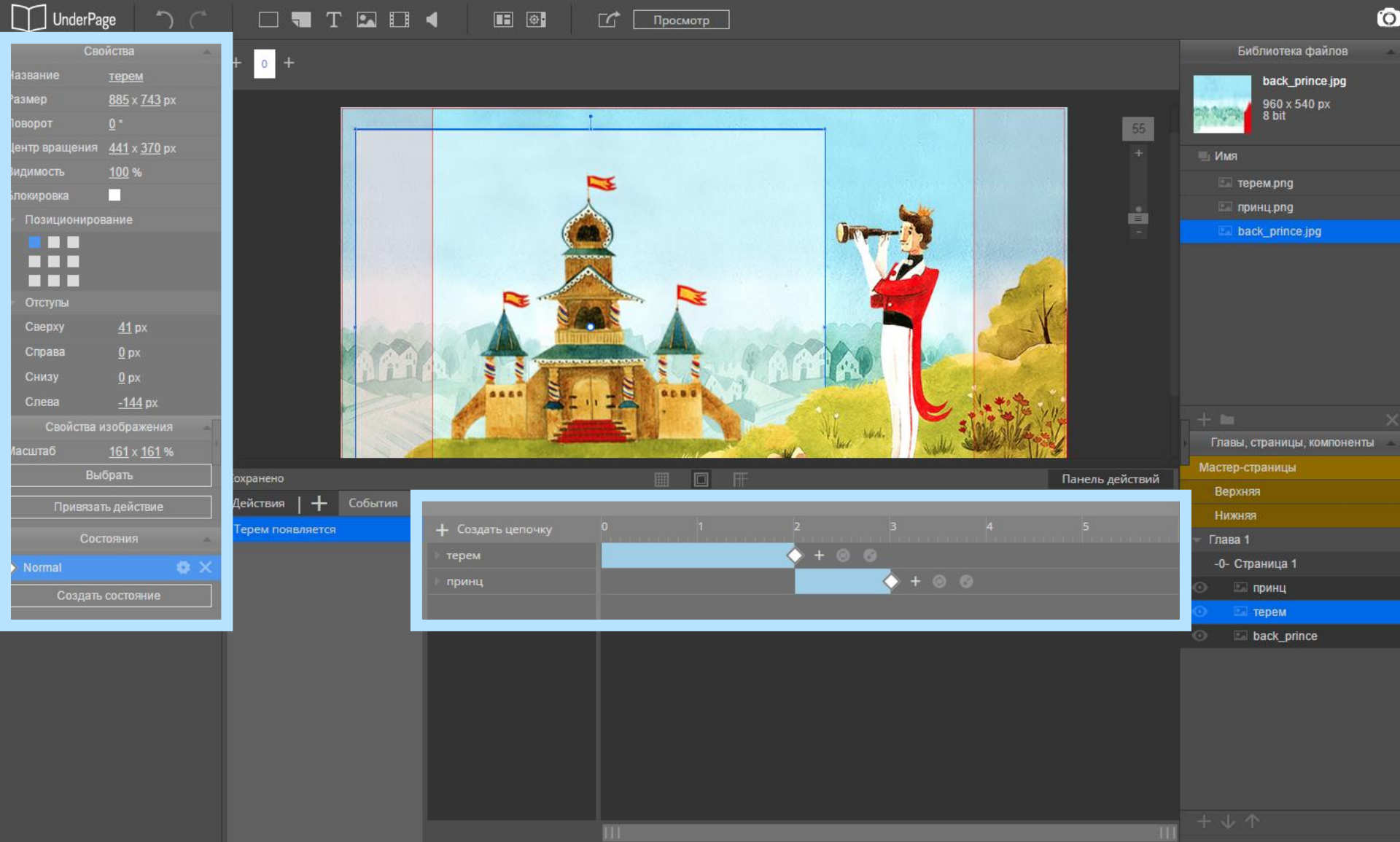
<внутренний блок 2 (атрибуты...)>

<поле 1>

<поле 2>



Полигон для испытаний



Полигон для испытаний

<up-inspector>

```
<up-inspector label="Свойства"...
```

```
<up-inspector-list label="Позиционирование"
```

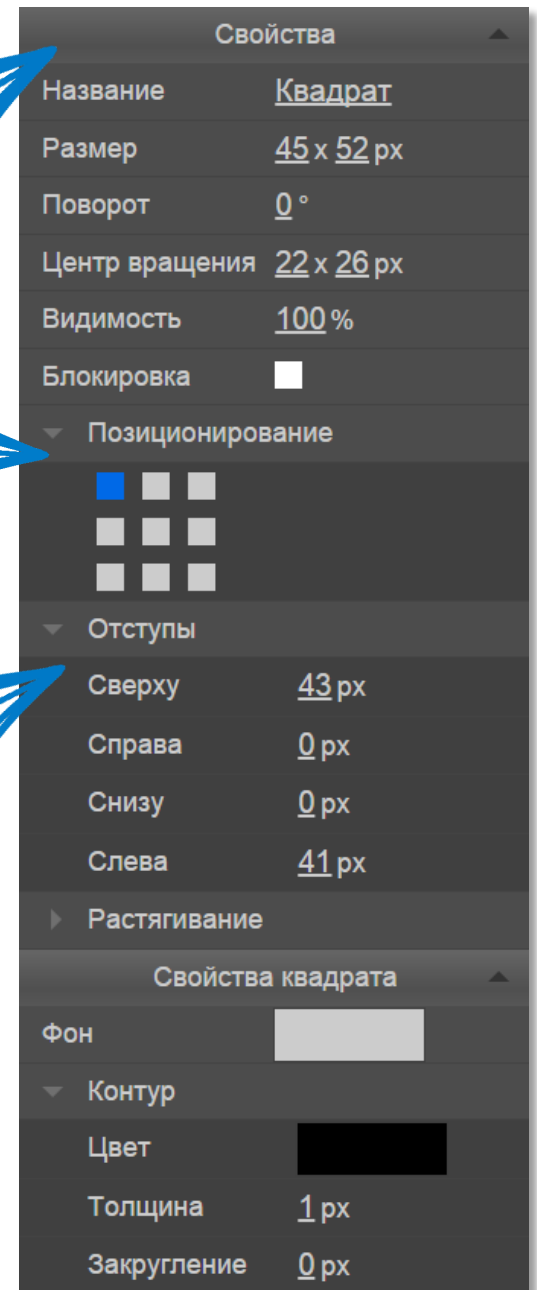
```
<!-- элементы управления -->
```

```
<!-- элементы управления -->
```

```
<up-inspector-list label="Отступы"
```

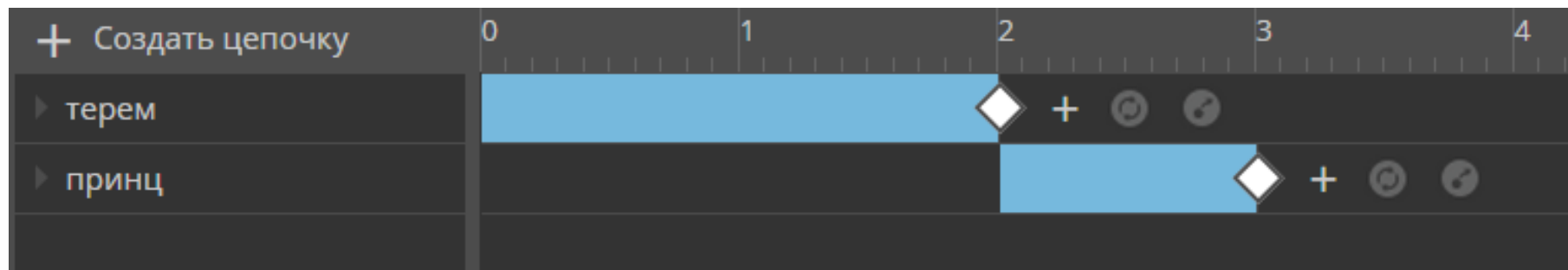
```
<!-- элементы управления -->
```

```
<!-- элементы управления -->
```

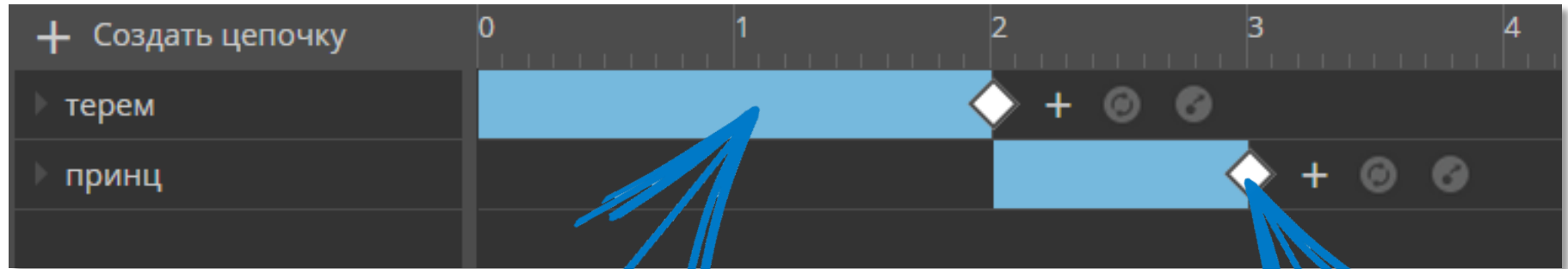


```
<up-inspector name="properties" label="Свойства">
  <up-inspector-list name="position" label="Позиционирование">
    <!-- элементы управления -->
  </up-inspector-list>

  <up-inspector-list name="offset" label="Отступы">
    <!-- элементы управления -->
  </up-inspector-list>
</up-inspector>
```



<up-sausage>



```
<up-sausage type="full" inner-length="200" length="400" ...
```

```
<up-state-marker />
```

```
<up-sausage  
  type="{{type}}"  
  inner-length="{{innerLength}}"  
  length="{{length}}">  
  
  <up-state-marker />  
  
</up-sausage>
```

Резюме

- Web Components сильны на обоих уровнях
- Система превращается в набор логических блоков
- Полученный результат удобно поддерживать

Так здорово ли работать
с веб-компонентами
в реальном проекте?





Определенно, ДА!

Наиболее частые проблемы

- Не все документировано
- Возможны изменения API
- Мало информации на [stackoverflow](#) по свежим версиям библиотек (...но это временно)

Перспективы развития



Если вы начинаете прямо сейчас и прямо с Polymer



0.5

Много готовых компонентов



0.9

База API для 1.0

мы тут



1.0

Обещают уже летом!

0.5

0.8

4000ms

Производительность polyfills (меньше - лучше)

<https://github.com/polymerlabs/benchmarks>

3000

2000

1000

0

Desktop Chrome
Macbook Pro
4.1x Faster

Desktop Safari
Macbook Pro
5.3x Faster

Mobile Safari
iPhone 6
5.3x Faster

Desktop Firefox
Macbook Pro
8x Faster

webcomponents.org

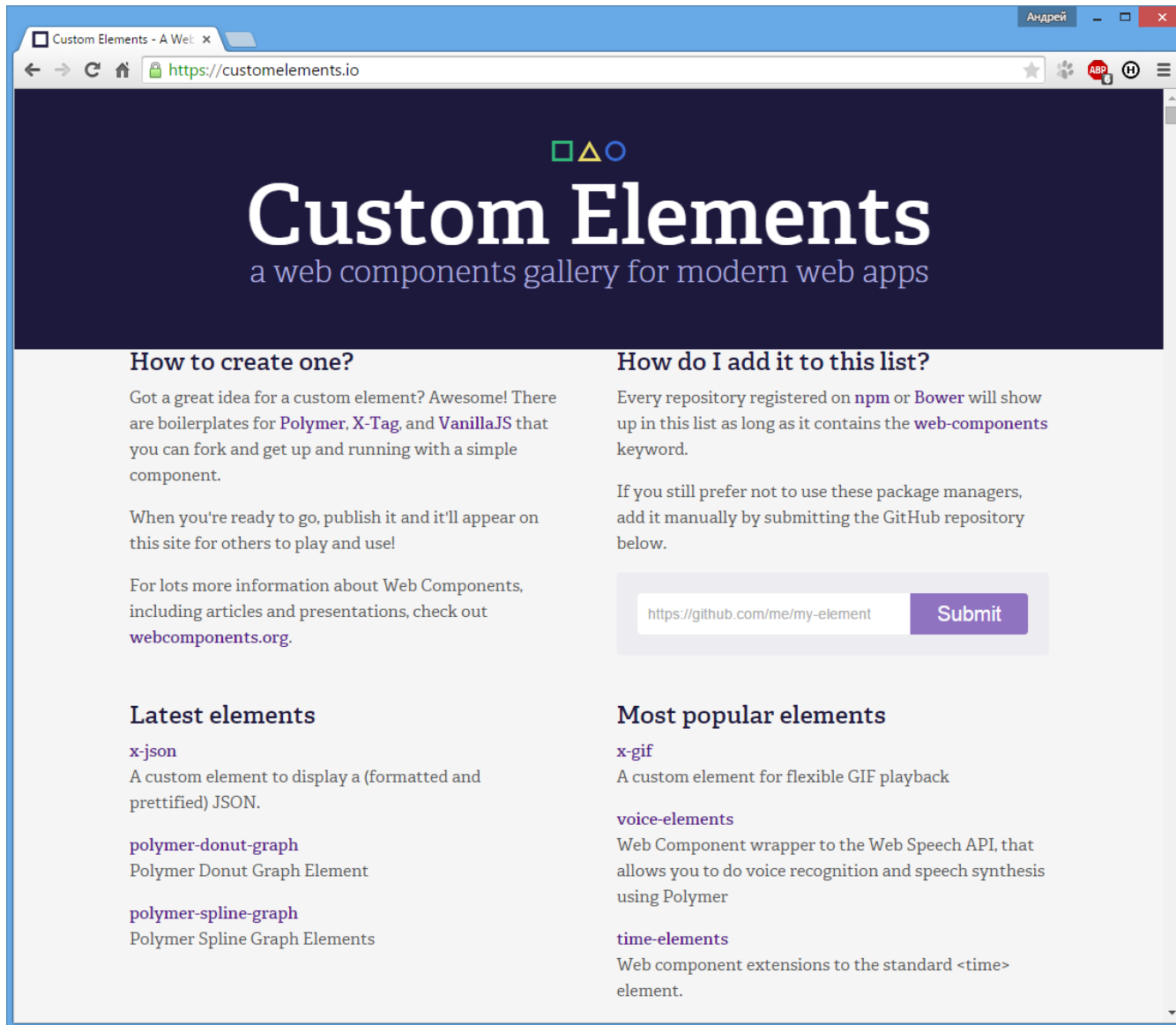


The screenshot shows the webcomponents.org website in a browser window. The browser's address bar displays 'webcomponents.org'. The website's navigation bar includes links for HOME, POLYFILLS, ARTICLES, PRESENTATIONS, PODCASTS, and RESOURCES, along with a Google Custom Search bar. The main header features the WebComponents.org logo, which is a stylized orange cube with a white wrench and screwdriver forming a circular shape. Below the logo, the text 'WebComponents.org' is prominently displayed, followed by the tagline 'a place to discuss and evolve web component best-practices'.

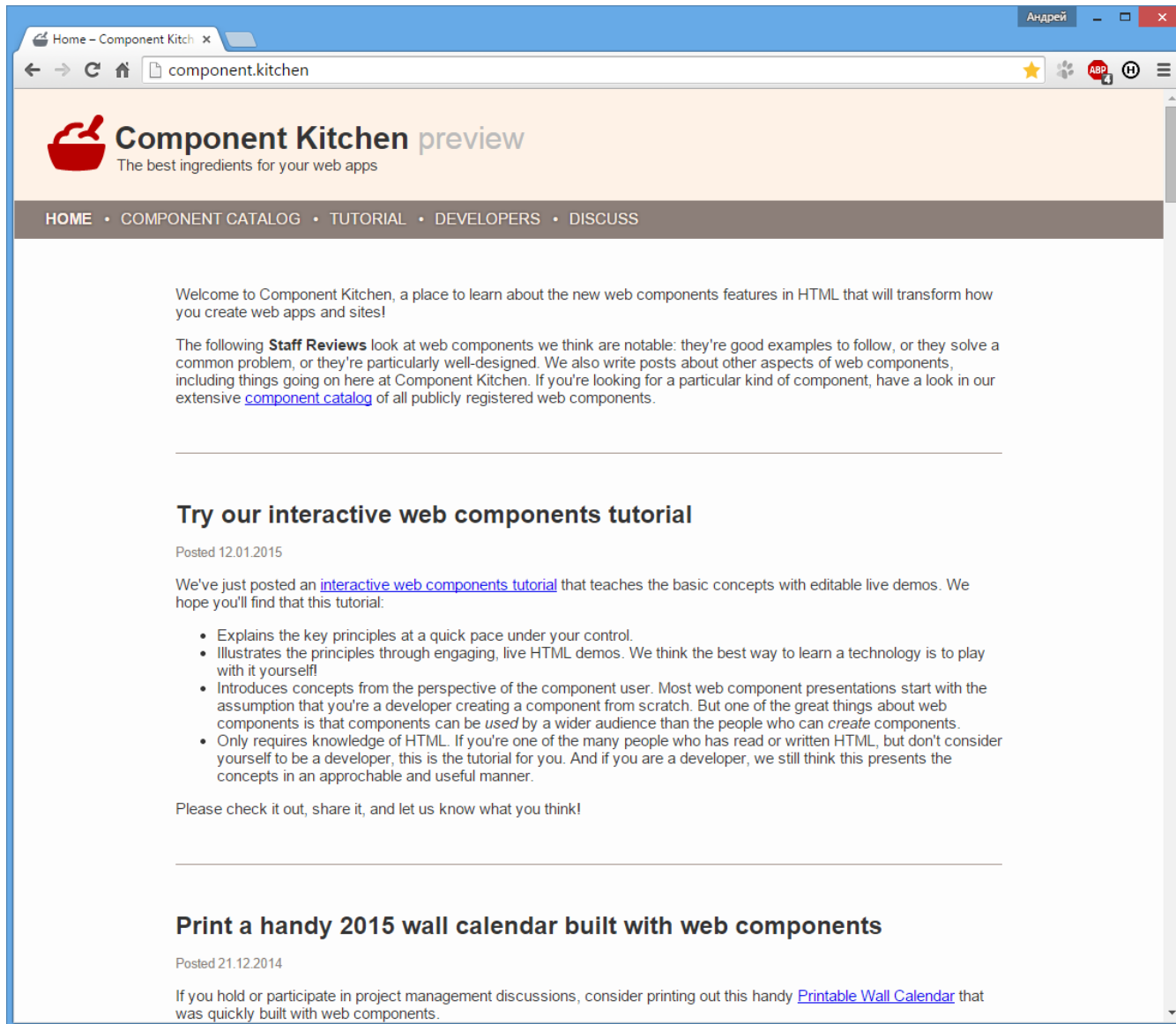
The website is organized into several sections:

- POLYFILLS**: This section explains that webcomponent.js polyfills enable Web Components in browsers that lack native support. It provides instructions on how to install the polyfills using Bower or npm, and includes a button to download the webcomponents.js file. The download information specifies version 0.6.1 (116KB minified, 34KB gzipped) and a link to learn more about the polyfills.
- DISCOVER**: This section highlights three key resources: 'CUSTOMELEMENTS.IO' (a gallery of Web Components created by the community), 'BUILT WITH POLYMER' (a curated collection of web apps and websites using Polymer), and 'COMPONENT KITCHEN' (a component catalog for a mainstream HTML audience).
- ARTICLES**: This section features an article titled 'WHY WEB COMPONENTS?'. The article discusses the promise of Web Components and the challenges of overselling. It mentions that while they may not solve all life's problems, they provide a paradigm shift from traditional web development. The article also addresses the confusion surrounding Web Components and encourages readers to read more.
- BROWSER SUPPORT**: This section lists the browsers supported by the polyfills: CHROME, OPERA, EDGE, SAFARI, and IE.
- SPECS**: This section includes a link to 'CUSTOM ELEMENTS'.

customelements.io



component.kitchen



Pluralsight - HTML5 Web Component Fundamentals

The screenshot shows the Pluralsight website interface. At the top, the browser address bar displays the URL `www.pluralsight.com/courses/web-components-shadow-dom`. The Pluralsight logo is on the left, with navigation links for 'Browse courses', 'Blog', and 'Teach'. A search bar, 'Subscribe' button, and 'Sign in' button are on the right. Below the navigation bar, there are filters for 'Top tags', 'Popular', 'New releases', and 'Products'. The main header area has a large orange banner with the course title 'HTML5 Web Component Fundamentals' and a subtitle 'Learn how to use the Shadow DOM, Custom Elements, Templates, and Imports to create reusable web components.' Below the banner, the course is attributed to 'Cory House'. A 'Table of contents' section is on the left, with an 'Expand all' link and a 'Start free trial now' button. The table lists 10 items with play icons and durations. On the right, the 'Course content' section lists 'Table of contents', 'Description', 'Exercise files', 'Assessment', and 'Discussion'. Below that, the 'More info' section shows the level as 'Beginner', a 5-star rating, a duration of '5h 3m', a release date of '10 Jan 2015', and a 'Features' section with a Creative Commons license icon. A vertical 'Feedback & Support' button is on the far right.

HTML5 Web Component Fundamentals

Learn how to use the Shadow DOM, Custom Elements, Templates, and Imports to create reusable web components.

by Cory House

Table of contents [Expand all](#) [Start free trial now](#)

▶ Five Problems, One Solution	23:52
▶ Templates	27:50
▶ Custom Elements	43:34
▶ Shadow DOM Fundamentals	34:19
▶ Shadow DOM Insertion Points & Events	53:34
▶ Shadow DOM Styling	51:34
▶ Imports	56:00
▶ Native Alternatives	10:43
▶ Resources	2:05

Course content

- ▶ Table of contents
- “” Description
- 📄 Exercise files
- 📝 Assessment
- 💬 Discussion

More info

Level **Beginner**

Rating ★★★★★

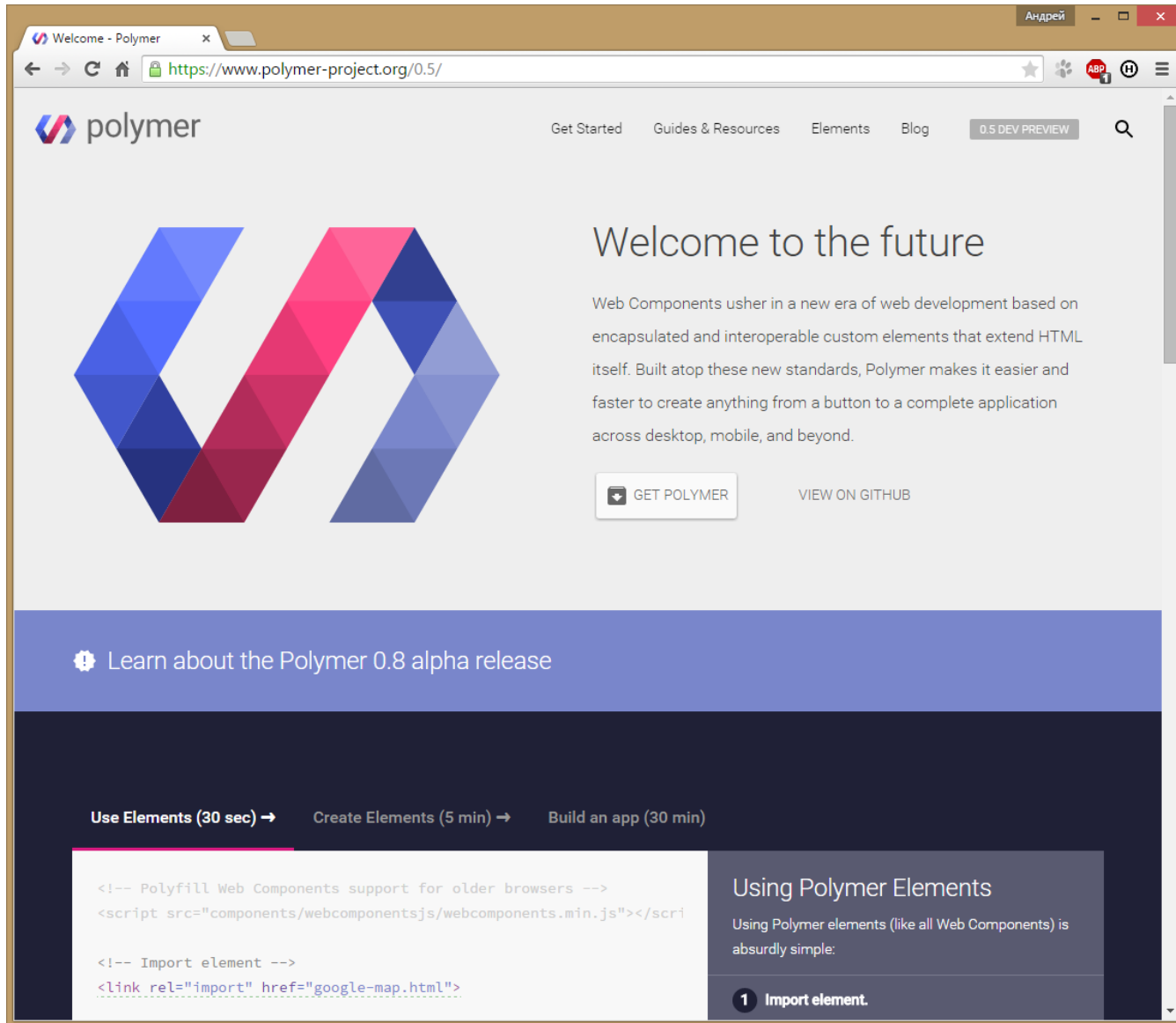
Duration **5h 3m**

Released **10 Jan 2015**

Features

Feedback & Support

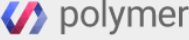
polymer-project.org



The screenshot shows the Polymer Project website in a web browser. The browser's address bar displays `https://www.polymer-project.org/0.5/`. The website's header includes the Polymer logo, navigation links for "Get Started", "Guides & Resources", "Elements", and "Blog", and a "0.5 DEV PREVIEW" badge. The main content area features a large, colorful geometric logo on the left and a "Welcome to the future" heading on the right. Below the heading, a paragraph describes Web Components and Polymer's role. Two buttons, "GET POLYMER" and "VIEW ON GITHUB", are positioned below the text. A blue banner below the main content area contains a link to "Learn about the Polymer 0.8 alpha release". At the bottom, a dark blue section contains a progress bar with three steps: "Use Elements (30 sec) →", "Create Elements (5 min) →", and "Build an app (30 min)". The first step is highlighted. Below the progress bar, a code block shows HTML snippets for polyfilling and importing elements. To the right of the code block, a section titled "Using Polymer Elements" explains that using Polymer elements is simple and lists "1 Import element." as the first step.

Welcome - Polymer x

← → ↻ 🏠 🔒 `https://www.polymer-project.org/0.5/` ★ 🐾 🔒 🔒 🔒

 polymer

Get Started Guides & Resources Elements Blog 0.5 DEV PREVIEW 🔍



Welcome to the future

Web Components usher in a new era of web development based on encapsulated and interoperable custom elements that extend HTML itself. Built atop these new standards, Polymer makes it easier and faster to create anything from a button to a complete application across desktop, mobile, and beyond.

📦 GET POLYMER VIEW ON GITHUB

🔔 Learn about the Polymer 0.8 alpha release

Use Elements (30 sec) → Create Elements (5 min) → Build an app (30 min)

```
<!-- Polyfill Web Components support for older browsers -->
<script src="components/webcomponentsjs/webcomponents.min.js"></script>

<!-- Import element -->
<link rel="import" href="google-map.html">
```

Using Polymer Elements

Using Polymer elements (like all Web Components) is absurdly simple:

- 1 Import element.

Резюме

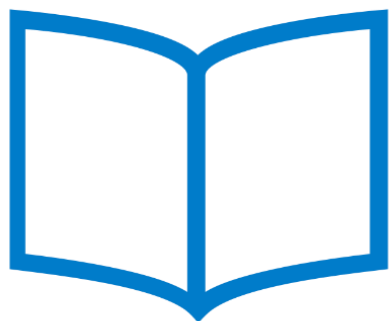
- Интеграция в реальный проект – уже возможна
- Инструменты – активно развиваются
- Библиотеки уже сегодня имеют широкие возможности
- Вы получаете модульность, о которой можно только мечтать!

Андрей Рахманов
rahmanov@enaza.ru

Дмитрий Чертков
chertkov@enaza.ru

Проект UnderPage

goo.gl/McfoKL



UnderPage

Презентация

goo.gl/qOuhYP



Попробуйте Web Components
Это вкусно!